ConvertPro - Full Technical Documentation

1. Project Overview

ConvertPro is a high-performance, full-stack web application designed for fast, reliable, and secure image processing. It allows users to upload, convert, optimize, and edit images entirely in their browser while leveraging a powerful backend processing engine.

The application was recently streamlined to focus purely on high-utility tools, prioritizing stability and massive file handling capabilities over bloated, unstable features.

---

2. Technology Stack

Frontend (User Interface)

- **Framework:** React.js (built with Vite for lightning-fast compilation).
- **Styling:** TailwindCSS (for responsive, modern, utility-first styling).
- **Routing:** React Router v6 (for seamless, single-page application navigation).
- **Icons:** Lucide-React (for clean, consistent vector icons).
- **HTTP Client:** Axios (for handling complex `multipart/form-data` file uploads to the backend).

Backend (Processing Engine)

- **Server:** Node.js with Express.js framework.
- **File Handling:** `multer` middleware (configured to handle massive file streams in memory and disk).
- **Image Processing Engine:** `sharp` (A blazingly fast C++ based image processing library used for resizing, cropping, converting, and compressing images).
- **Archiving:** `archiver` (Used to instantly zip multiple processed images together for a single batch download).
- **PDF Generation:** `pdfkit` (Used to seamlessly compile multiple raster images into a single PDF document).

---

3. Core Architecture & Workflows

The Frontend Hub (`ToolPage.jsx`)

The entire application operates through a central dynamic engine called `ToolPage.jsx`. Instead of having 20 different web pages for 20 different tools, `ToolPage` dynamically morphs its UI based on the URL (e.g., `/tool/compress` or `/tool/convert`).

1. **Upload Interception:** It instantly rejects unsupported proprietary formats (like Apple's `.heic`) to protect the server from crashing.
2. **Payload Construction:** It takes the user's files and builds a `FormData` payload, ensuring the files are always attached under the `images` key.
3. **Download Management:** Once the backend responds with a raw binary Blob, it dynamically determines the correct file extension (`.png`, `.zip`, `.pdf`) and forces the browser to download it.

The Backend Engine

The backend is split into three main micro-routes to keep the code clean:

1. **`/api/convert` (`routes/convert.js`):** The heavy lifter. It loops through arrays of images, uses `sharp` to transcode them into formats like WEBP, TIFF, or AVIF, and zips them up if there are multiple files.
2. **`/api/edit` (`routes/edit.js`):** Handles geometrical transformations (Crop, Resize, Rotate, Flip).
3. **`/api/pdf` (`routes/pdf.js`):** Specifically handles stitching multiple images together onto A4-sized PDF pages using `pdfkit`.

---

4. Server Guardrails & Technical Limitations

To ensure the application can run on free or low-cost cloud hosting (like Render or Heroku) without crashing from memory exhaustion, strict mathematical limits were engineered into the backend `multer` configuration:

- **Maximum Individual File Size:** `100 MB`
- **Maximum Files Per Batch:** `20 files`

Why these limits?
A standard free-tier cloud server has 512MB of RAM. If a user was allowed to upload 100 files at 100MB each, the server would receive a 10GB payload and instantly crash. By capping the batch limit to 20 files, we ensure the absolute maximum payload is 2GB, keeping the server stable while still allowing massive, professional-grade image processing.

---

5. Security & Privacy

- **Zero Retention:** The backend does not connect to a database or AWS S3. All files are temporarily stored in the `/uploads` directory for processing and are automatically purged.
- **Metadata Stripping:** When images are converted using `sharp`, EXIF metadata (like GPS location coordinates from smartphones) is automatically stripped for user privacy.